

Debugging Strategies

Project: Transparent Book Recommender

1. Your mission

Your learning target

I can design revealing tests for a function.

I can compare expected and actual results.

I can repair a defect and rerun earlier cases.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

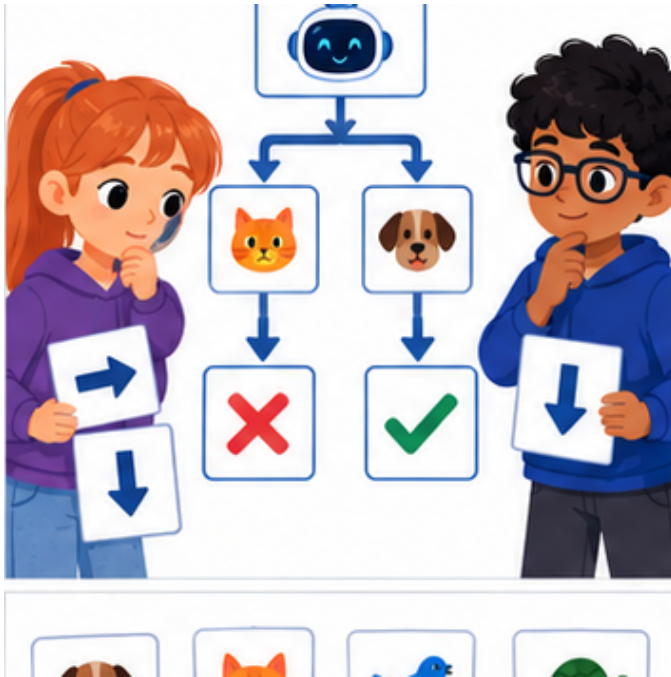
Without opening the old lesson, explain one key idea from Loops and Stopping. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Branching Logic. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

Week 6: Debugging Strategies

Good debugging isolates a cause with controlled tests.



THINKER MISSION

Use divide-and-check to locate a faulty branch.

Thinker name and date:

Write or draw your reasoning and evidence.

Readiness for the connected explanation

For capacity 9 and used 2, what should remaining return?

This week's end goal

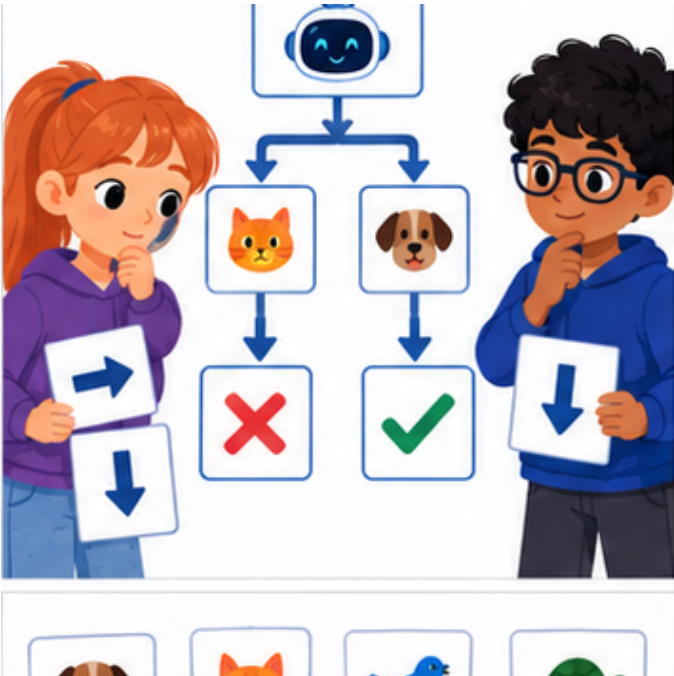
By the end, I can test a function with expected values that distinguish a particular bug.

2. Notice the evidence

PICTURE STORY

Notice → Predict → Explain

Begin with visible evidence. Do not explain more than the picture and information support.



MYSTERY

The learners must use debugging strategies to solve a problem. What information do they have? What decision or calculation do they make? What would count as evidence that it worked?

Three observations (what you can point to):

Write or draw your reasoning and evidence.

My prediction and the clue supporting it:

Write or draw your reasoning and evidence.

A closer explanation

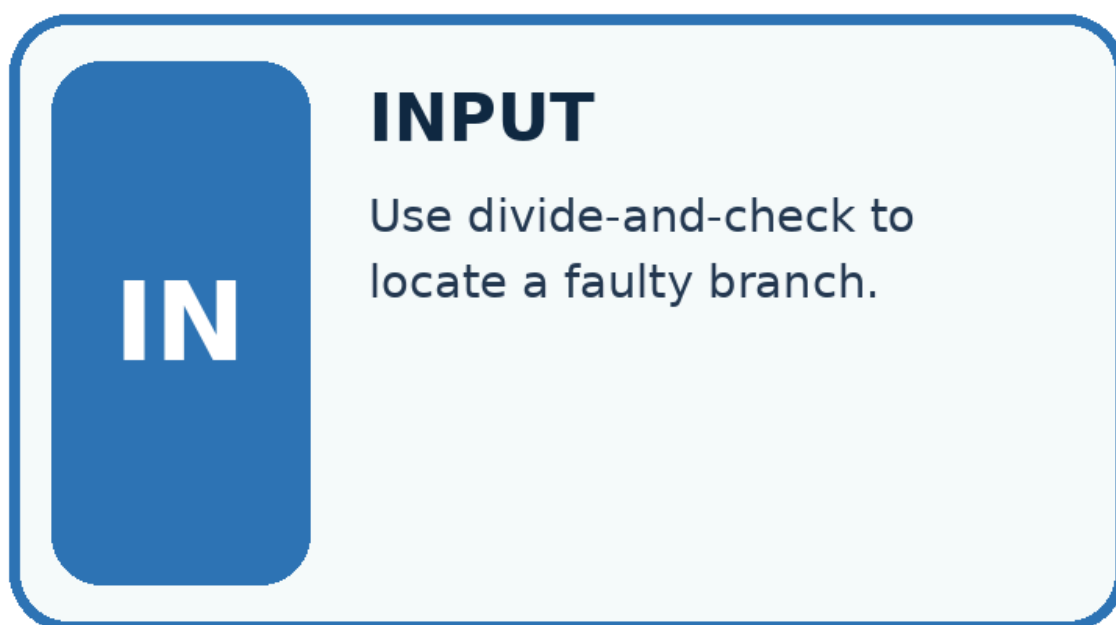
A function can pass one example by coincidence. Start with a written rule, calculate expected results independently, and include ordinary, zero and boundary cases. We will test a subtraction function under the contract $0 \leq \text{used} \leq \text{capacity}$.

3. See how it works

VISUAL MECHANISM

Input → Process → Output

Trace the information. At each stage, ask what changed and how you could test it.



↓



PROCESS

Good debugging isolates a cause with controlled tests.



OUT

OUTPUT

Use divide-and-check to locate a faulty branch.

Which stage could fail even when the other two are correct? Explain:

Write or draw your reasoning and evidence.

A closer explanation

```
def remaining(capacity, used):  
    return capacity - used  
  
cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]  
  
for capacity, used, expected in cases:  
    actual = remaining(capacity, used)  
    print(actual == expected)
```

Follow the evidence

Step or case · Evidence · Meaning

Step or case: Ordinary case

Evidence: 8 and 3

Meaning: Expect 5

Step or case: Full case

Evidence: 8 and 8

Meaning: Expect 0

Step or case: Unused case

Evidence: 8 and 0

Meaning: Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result. The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with AssertionError.

Give each test a job

A useful test includes an input and an answer calculated from the written rule before the program runs. For 8 spaces with 3 used, expect 5 left. For 8 used, expect 0. For none used, expect 8. These cases ask different questions. An incorrect addition rule happens to pass the unused case because adding zero and subtracting zero give the same result. That is why one passing example is weak evidence.

Connect the picture and the explanation

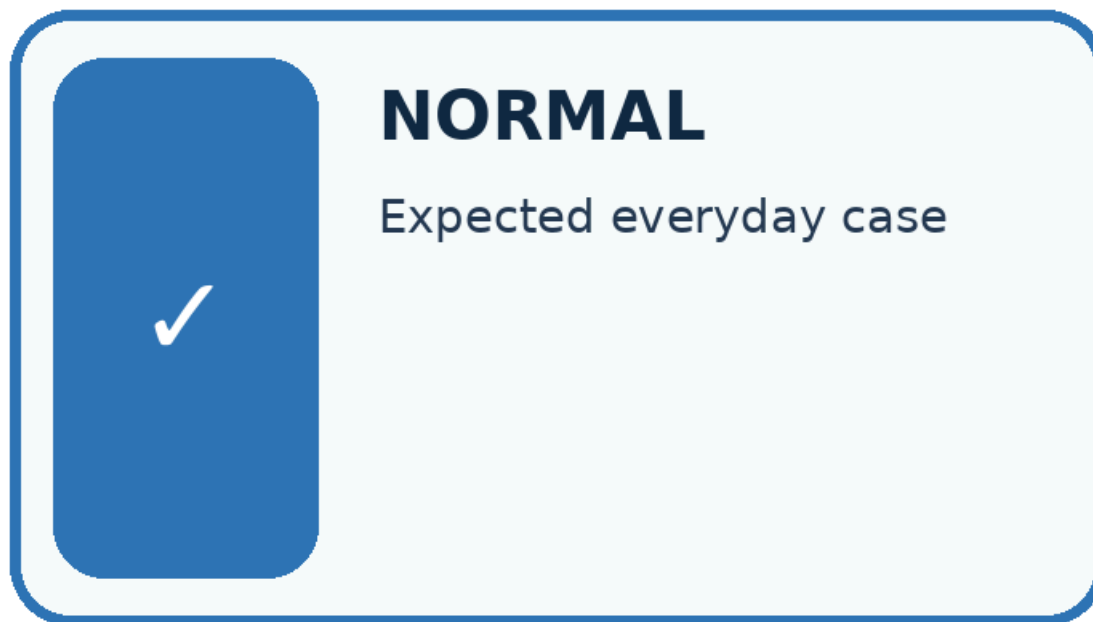
Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

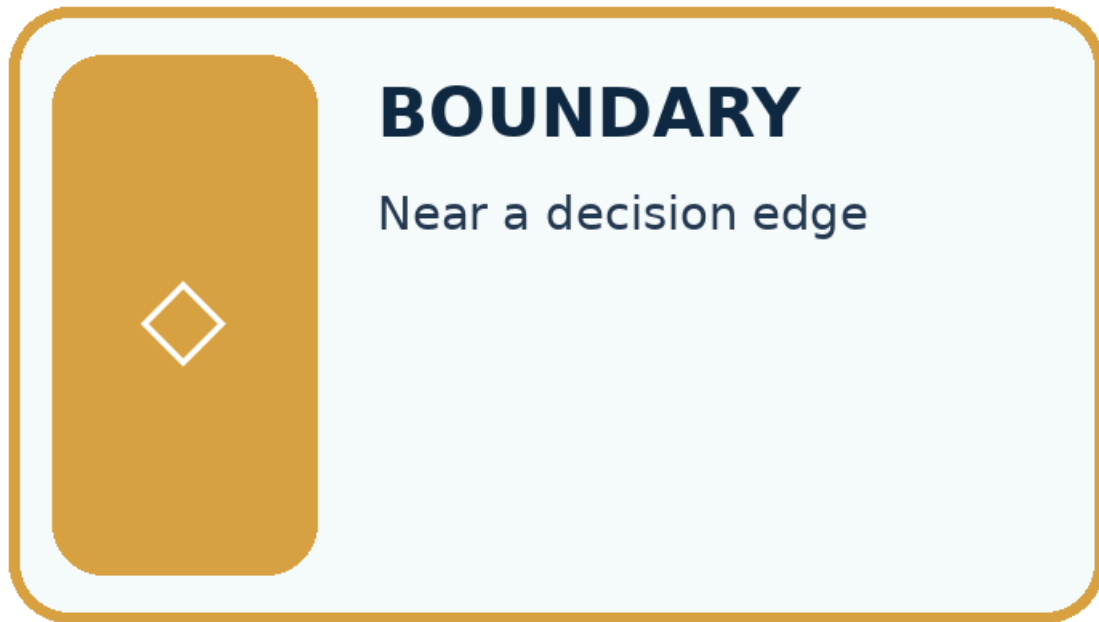
4. Compare the cases

CASE GALLERY

Four Tests, Four Kinds of Evidence

A system is not understood until it survives more than one comfortable example.

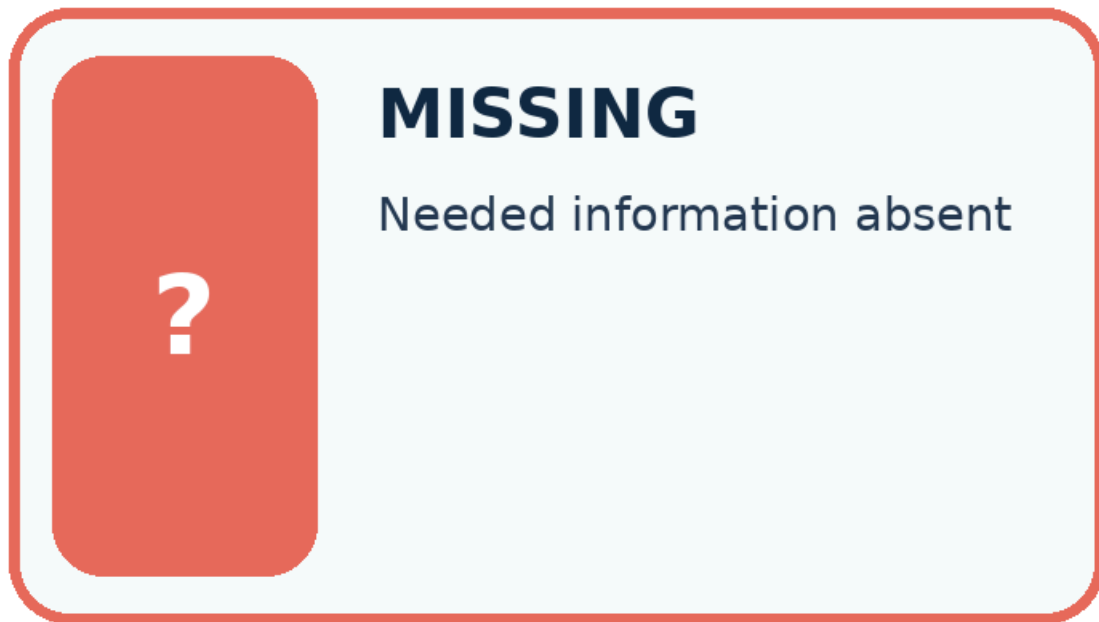




BOUNDARY

Near a decision edge

NORMAL CASE: predict and name evidence. BOUNDARY CASE: predict and name evidence.



MISSING

Needed information absent



MISSING INFORMATION: predict and name evidence. TRANSFER CASE: predict and name evidence.

Which case is most likely to reveal a hidden assumption? Why?

Write or draw your reasoning and evidence.

Change one thing in the pictured case

Change: Repair the body to $\text{bundle_count} \times 6 + \text{loose}$.

Prediction: Expected results for (2,1),(0,3),(2,0) are 13,3,12.

Reason: Only bundles are groups of six; loose books are counted individually.

5. Words that unlock the idea

VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.

1

BUG

a cause of incorrect
behavior

2

DEBUG

find and repair a cause

3

CONTROLLED TEST

change one factor at a time

4

RETEST

run the same test after a change

Word · My own example · Not this / boundary

Word: bug

My own example: A defect that makes a program behave differently from its intended specification.

Not this / boundary: _____

Word: debug

My own example: Find and repair the cause of a program's incorrect behaviour.

Not this / boundary: _____

Word: controlled test

My own example: A test with specified inputs, conditions and expected measurements.

Not this / boundary: _____

Word: retest

My own example: Run a test again after a change or under specified new conditions.

Not this / boundary: _____

Words used in the closer explanation

Word · Meaning

Word: Test case

Meaning: Inputs and an expected result

Word: Boundary

Meaning: An edge of a permitted range

Word: Regression

Meaning: A previously working case that fails after a change

Word: Assertion

Meaning: A check that raises an error when its condition is false

6. Follow a worked example

WORKED EVIDENCE

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete evidence.

WEEK 6 • WORKED EVIDENCE

- 1 Observed: wet-weather case recommends sandals.
- 2 Divide: input correct? yes. rain branch reached? no.
- 3 Cause: condition says rain = false instead of true.
- 4 Fix that condition only; rerun the original case and a dry-weather case.

1. Observed: wet-weather case recommends sandals.
2. Divide: input correct? yes. rain branch reached? no.
3. Cause: condition says rain = false instead of true.
4. Fix that condition only; rerun the original case and a dry-weather case.

Recalculate, retrace, or restate the reasoning in your own representation:

Write or draw your reasoning and evidence.

CHECK Did the evidence answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

Would replacing the body with return 5 pass every worked case? Identify the first revealing test.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debugging Strategies

A fictional shelf function should count books in complete bundles plus loose books.

The facts we will use

Correct definition: `shelf(bundle_count,loose)` returns `bundle_count*6+loose`.

Faulty copied body returns `(bundle_count+loose)*6`. Predict tests before running.

AI ACADEMY / AI-2/6

Expected outputs versus a specific bug

	Bundles	Loose	Correct	Faulty
Test 1	2	1	13	18
Test 2	0	3	3	18
Test 3	2	0	12	12

Read a cell using both its row and column labels.

Debugging Strategies

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Expected normal test

For (2,1), correct result is $2 \times 6 + 1 = 13$.



2. Faulty trace

Copied code gives $(2+1) \times 6 = 18$.



3. Zero-bundle test

For (0,3), correct 3; copied code 18.



4. Nonrevealing test

For (2,0), both return 12; a pass does not locate every bug.

CHANGE ONE THING

Repair the body to `bundle_count*6+loose`.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Expected normal test

What happens and why: For (2,1), correct result is $2 \times 6 + 1 = 13$.

Step: 2. Faulty trace

What happens and why: Copied code gives $(2+1) \times 6 = 18$.

Step: 3. Zero-bundle test

What happens and why: For (0,3), correct 3; copied code 18.

Step: 4. Nonrevealing test

What happens and why: For (2,0), both return 12; a pass does not locate every bug.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Test cases need not all expose the same error. Here loose books must be added after multiplying bundles. A zero-loose case cannot distinguish the two formulas because multiplying zero loose books changes nothing. Keep expected and actual columns so an observed output does not quietly become its own expected answer.

What this example does not establish

These three tests check a specified formula and bug; they do not prove correct handling of every type or invalid request.

Read the labelled picture: Which test most directly isolates the handling of loose books, and what should it produce?

Change one thing: Repair the body to `bundle_count*6+loose`. Predict the result before checking the explanation. What remains the same?

7. Find and repair a mistake

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak evidence.



COMPARE • DIAGNOSE

Weak idea: Changing several steps at once makes debugging faster. Strong move: locate evidence, change one cause, and retest.

Evidence record · Expected · Actual · Likely cause / next test

Evidence record: normal case

Expected: works

Actual: _____

Likely cause / next test: _____

Evidence record: boundary case

Expected: stated rule

Actual: _____

Likely cause / next test: _____

Evidence record: missing input

Expected: safe response

Actual: _____

Likely cause / next test: _____

Evidence record: fresh transfer

Expected: generalizes

Actual: _____

Likely cause / next test: _____

Repair one cause. Why is this repair better than changing several things?

Write or draw your reasoning and evidence.

A closer explanation

Passing this small suite supports these particular cases. It neither proves every allowed input correct nor checks invalid input handling. A test outside the contract needs a separately specified expected behavior.

All three stock tests pass. Does that prove correct handling of negative counts?

8. Practise together

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.



TRACE THE LOGIC

Predict the result. Trace every condition or calculation. Compare expected with actual. Explain the first difference.

SET evidence_record PREDICT expected_result RUN one normal case and one boundary case
COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST
EXPLAIN result, limitation, and next safe step

Trace one input. Which line changes the state or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

A faulty return capacity + used gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases. Never change an expected result merely to match faulty output.

Practise with less help: Find a revealing case

Use workbook W2 for the connected example “Testing and debugging functions”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Which test most directly isolates the handling of loose books, and what should it produce?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



PREDICT → TEST → RECORD → EXPLAIN

Use normal, boundary, and unexpected test cards. Mark the first step where expected and actual behavior differ; change one cause; rerun all three.

Use normal, boundary, and unexpected test cards. Mark the first step where expected and actual behavior differ; change one cause; rerun all three.

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

Test / input: normal

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: boundary

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: missing

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

What did the hardest test teach you that the normal test did not?

Write or draw your reasoning and evidence.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce evidence without copying the worked example.



MASTERY LADDER

1 Name it • 2 Trace it • 3 Apply it • 4 Challenge it • 5 Explain limits

- 1 • NAME: Define the concept without repeating the textbook.
 - 2 • TRACE: Show the information, state, branch, calculation, or evidence path.
 - 3 • APPLY: Solve a different example independently.
 - 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.
 - 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.
- My strongest evidence of independent understanding:

Write or draw your reasoning and evidence.

11. Connect your project

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



TRANSPARENT BOOK RECOMMENDER

When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

THIS WEEK'S CHECKPOINT When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

SYSTEM MAP Reader need → allowed inputs → transparent rule or score → ranked candidates → explanation → test record → human decision

What will you add, change, or verify in the project this week?

Write or draw your reasoning and evidence.

What should remain under human control?

Write or draw your reasoning and evidence.

RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits. Never confuse a ranking with a fact about a person.

Evidence for your project

Test the recommender’s helper functions with normal, zero and boundary inputs before debugging the whole application.

12. Show what you know

INDEPENDENT MASTERY PROOF

Explain • Transfer • Test • Defend

Complete this page without copying the worked example. Your reasoning matters more than decoration.



INDEPENDENT PROOF

Use divide-and-check to locate a faulty branch.

CHALLENGE Use divide-and-check to locate a faulty branch.

1. My fresh example, trace, calculation, or design:

Write or draw your reasoning and evidence.

2. My evidence that it works (include a boundary or missing-data case):

Write or draw your reasoning and evidence.

3. One limitation, fairness concern, or situation needing human review:

Write or draw your reasoning and evidence.

SELF-CHECK ☐ I explained the mechanism ☐ I used evidence ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

13. Study a complete case

Week 6 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

Award two points per matching trait. A bug awards two for every trait.

The question

Test one match and one nonmatch.

Worked explanation

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

Explain it in your own words

Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

Trace the supplied case visually

Put the addition inside the condition

Evidence or stage · Value or result

Evidence or stage: One matching trait

Value or result: +2

Evidence or stage: One nonmatching trait

Value or result: +0

Evidence or stage: Expected total

Value or result: 2

Evidence or stage: Buggy total

Value or result: 4 because both were awarded points

Award two points per matching trait. A bug awards two for every trait.

Test one match and one nonmatch.

Expected 2; buggy result 4. Put the addition inside the match condition. Test zero, one and two matches for 0,2,4.

14. Try a new case independently

Independent case: Test a new function independently

Define stock(packs, loose) to return packs * 4 + loose. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty packs + loose body.

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Week 6 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

A loop should run four times but runs five. You change its colour and stopping condition at once. Which single change is relevant to test first?

My answer, with a trace, calculation or supporting evidence

Write or draw your reasoning and evidence.

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed input and expected result

Write or draw your reasoning and evidence.

Connect to your project

When a ranking surprises you, inspect feature values, weights, arithmetic, and sorting separately.

The evidence I will keep

Write or draw your reasoning and evidence.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice evidence together.

Transfer practice from the original programme

Intended score is 3 per matching trait. Bug: score starts at 3 and adds 3 for each match. Give expected and actual results for 0,1,2 matches and repair the cause.

Explain the mechanism, evidence and one limit.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.

15. Your lesson route

Start with this question: A fictional shelf function should count books in complete bundles plus loose books.

By the end, I can · How I will show it

By the end, I can: Design revealing tests for a function

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Compare expected and actual results

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Repair a defect and rerun earlier cases

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project evidence. Your teacher may spread this lesson across several classes.

Keep your first prediction. Compare it with what the evidence shows and explain any change.

16. Finish and return

Close your notes. Explain this goal in your own words: Design revealing tests for a function

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an input or assumption and justify your prediction.

Save your workbook evidence. At the next review, try a different case before opening your old answer.